MySupplyCo – An Integrated Digital Platform for Inventory Status Information and User Support in Civil Supply Outlets

A Mini Project Report

submitted to the **APJ Abdul Kalam Technological University**

in partial fulfillment of the requirements for the award of degree

Bachelor of Technology

in

Computer Science and Engineering

by

ABHINAND ULLAS P(TCR23CS004)

IKRAMUL HAQUE TK(TCR23CS043)

SAGAR P(TCR23CS060)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

GOVERNMENT ENGINEERING COLLEGE, THRISSUR

KERALA

MARCH 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
GOVERNMENT ENGINEERING COLLEGE
THRISSUR – 680009

CERTIFICATE

This is to certify that the report entitled **MySupplyCo – An Integrated Digital Platform for Inventory Status Information and User Support in Civil Supply Outlets** submitted by **ABHINAND ULLAS P (TCR23CS004)**, **IKRAMUL HAQUE TK (TCR23CS043)**, & **SAGAR P (TCR23CS060)** to the APJ Abdul Kalam Technological University in partial fulfillment of the B.Tech. degree in Computer Science and Engineering is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Dr. Gilesh M P
(Project Guide)
Professor
Department of CSE
GEC Thrissur

Mr. George Mathew
(Project Coordinator)
Assistant Professor
Department of CSE
GEC Thrissur

Dr. Swaraj K P
Head of the Department
Department of CSE
GEC Thrissur

Place: Thrissur
Date: March 31, 2026

DECLARATION

We hereby declare that the mini project report **MySupplyCo – An Integrated Digital Platform for Inventory Status Information and User Support in Civil Supply Outlets**, submitted for partial fulfillment of the requirements for the award of degree of Bachelor of Technology of the APJ Abdul Kalam Technological University, Kerala is a bonafide work done by us under supervision of **Dr. Gilesh M P**.

This submission represents our ideas in our own words and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that we have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Thrissur
March 31, 2026

ABHINAND ULLAS P

IKRAMUL HAQUE TK

SAGAR P

ABSTRACT

The Kerala State Civil Supplies Corporation (SupplyCo) operates over 2000 retail outlets across Kerala, distributing essential commodities such as rice, sugar, cooking oil, and pulses to the public at subsidized rates. Despite its critical role in public welfare, citizens currently lack any reliable mechanism to check the availability of stock at their nearest outlet before making a physical visit. This information gap leads to wasted trips, long queues, and unnecessary frustration, disproportionately affecting elderly and rural populations.

This project presents **MySupplyCo**, an integrated digital platform that provides real-time inventory status information for civil supply outlets through a user-friendly cross-platform mobile application. The system employs a three-layer architecture comprising a government data simulator built with Django and PostgreSQL, a main backend server powered by Supabase (with PostgreSQL, authentication services, and serverless edge functions), and a cross-platform mobile application developed using the Flutter framework.

The platform enables citizens to search for SupplyCo stores by name, filter by district, or discover nearby outlets using GPS-based geolocation. Users can view real-time stock information including item availability, quantity, and pricing. The system features automated push notifications via Firebase Cloud Messaging (FCM) that alert users when out-of-stock items are restocked at their preferred store. A WhatsApp alert subscription interface provides an accessible channel for users less comfortable with mobile applications, particularly elderly citizens.

Key technical features include offline caching for stock data resilience during network disruptions, incremental data synchronization using checkpoint-based polling, database webhook-triggered restock notifications, multi-language localization supporting eight Indian languages (English, Malayalam, Hindi, Gujarati, Marathi, Bengali, Tamil, and Telugu), and secure authentication via OTP-based phone verification and Google OAuth.

The architecture is intentionally designed with a read-only security model, ensuring no direct access to government databases while maintaining data freshness through periodic API-based synchronization. This modular design allows seamless integration with official government APIs when they become available, making the system both a functional prototype and a deployment-ready framework for statewide rollout.

ACKNOWLEDGEMENT

We take this opportunity to express our deepest sense of gratitude and sincere thanks to everyone who helped us to complete this work successfully. We express our sincere thanks to **Dr. Swaraj K P**, Head of Department, Computer Science and Engineering, Government Engineering College, Thrissur for providing us with all the necessary facilities and support.

We would like to express our sincere gratitude to **Mr. George Mathew**, Assistant Professor, Department of Computer Science and Engineering, Government Engineering College, Thrissur for the support and co-operation.

We would like to place on record our sincere gratitude to our project guide
Dr. Gilesh M P, Professor, Computer Science and Engineering, Government Engineering College, Thrissur for the guidance and mentorship throughout this work.

Finally we thank our family, and friends who contributed to the successful fulfilment of this project work.

ABHINAND ULLAS P
IKRAMUL HAQUE TK
SAGAR P

Contents

Abstract **i**

Acknowledgement **ii**

Abbreviations **vii**

1 Introduction **1**

1.1 Topic Introduction 1

1.2 Problem Statement 2

1.3 Objectives 3

1.4 Novelty of Idea and Implementation Steps 4

1.4.1 Novelty 4

1.4.2 Implementation Steps 5

1.5 Societal and Industrial Relevance 6

1.5.1 Alignment with Digital India 6

1.5.2 Citizen Empowerment 6

1.5.3 Environmental Impact 6

1.5.4 Scalable Framework for Government Services 7

1.5.5 Industrial Relevance 7

2 Literature Review **8**

2.1 Introduction 8

2.1.1 Study Reports 8

2.1.2 Existing Systems 9

2.1.3 Future Directions and Research Opportunities 11

2.2 Conclusions and Gap Analysis 11

2.2.1 Conclusion 11

2.2.2 Gap Analysis 12

2.3 Summary 13

3 Feasibility Study and Requirements Analysis 14
3.1 Feasibility 14
3.1.1 Technical Feasibility 14
3.1.2 Economic Feasibility 15
3.1.3 Time Feasibility 16
3.1.4 Legal Feasibility 17
3.1.5 Operational Feasibility 17
3.2 Project Requirements 18
3.2.1 Implementation Requirements 18
3.2.2 Deployment Requirements 19

4 Project Design 20
4.1 Introduction 20
4.1.1 Purpose 20
4.1.2 Scope 20
4.1.3 Overview 20
4.2 System Architecture 21
4.2.1 Software Design 21
4.2.2 Architectural Overview 25

5 Implementation and Testing 29
5.1 Functionality Implementation and Test Cases 29
5.1.1 User Authentication Functionality 29
5.1.2 Store Discovery Functionality 30
5.1.3 Stock Viewing Functionality 32
5.1.4 Push Notification System 33
5.1.5 WhatsApp Alert Subscription 34
5.1.6 Localization (Multi-Language Support) 35
5.1.7 Profile and Settings 36
5.2 Testing Strategies 37
5.3 Source Code 38
5.4 Screenshots 38

6 Results and Discussion 43
6.1 Testing Results 43
6.2 User Feedback 45
6.3 Discussion 46

6.3.1 System Effectiveness 46
6.3.2 Security Model Validation 46
6.3.3 Offline Resilience 46
6.3.4 Scalability Potential 46
6.3.5 Limitations 47

7 Conclusions and Future Scope 48
7.1 Conclusion 48
7.2 Future Scope 49

References 51

List of Figures

4.1 System Architecture Diagram of MySupplyCo 21
4.2 Use Case Diagram 23
4.3 Data Flow Diagram 24
4.4 Application Navigation Flow 25

5.1 Splash Screen 39
5.2 Get Started / Onboarding Screen 39
5.3 Login / Register Screen 40
5.4 Home Page with Store Listing 40
5.5 Stock Page with Item Grid 41
5.6 WhatsApp Alert Subscription Page 41
5.7 Profile and Settings Page 42
5.8 Language Selection Dialog 42

ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
APK	Android Package Kit
ARB	Application Resource Bundle
BaaS	Backend as a Service
CLI	Command Line Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
ER	Entity–Relationship
ERP	Enterprise Resource Planning
FCM	Firebase Cloud Messaging
GPS	Global Positioning System
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
iOS	iPhone Operating System
JSON	JavaScript Object Notation
JWT	JSON Web Token
KTU	APJ Abdul Kalam Technological University
OAuth	Open Authorization
ORM	Object–Relational Mapping
OTP	One-Time Password
PDS	Public Distribution System
REST	Representational State Transfer
RPC	Remote Procedure Call
SDK	Software Development Kit
SMS	Short Message Service
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience

Chapter 1

Introduction

1.1 Topic Introduction

The Kerala State Civil Supplies Corporation Limited (SupplyCo), established in 1974, is one of the largest public distribution networks operating in India. With over 2000 retail outlets spread across all 14 districts of Kerala, SupplyCo serves as the backbone of the state's public distribution system, providing essential commodities such as rice, wheat, sugar, cooking oil, pulses, and other daily necessities at government-subsidized prices to millions of citizens [1].

The Public Distribution System (PDS) in India is a critical component of the government's food security framework, designed to ensure that essential commodities reach every household at affordable prices. In Kerala, SupplyCo plays a particularly vital role due to the state's high population density and its commitment to social welfare. The corporation operates under the Department of Food and Civil Supplies, Government of Kerala, and is responsible for procuring, storing, and distributing commodities through its extensive network of outlets.

Despite the critical importance of this distribution network, the current system operates with significant informational limitations. Citizens who wish to purchase commodities from a SupplyCo outlet have no reliable way to determine whether the items they need are currently in stock. The traditional method involves physically visiting the outlet, often traveling considerable distances, only to discover that the desired items are unavailable. This leads to wasted time, effort, and transportation costs, particularly affecting vulnerable populations such as the elderly, differently-abled individuals, and those living in rural areas.

The advent of smartphones and mobile internet has transformed how citizens interact with government services worldwide. India's Digital India initiative, coupled with Kerala's progressive stance on e-governance, has created an ecosystem ripe for digital transformation of public services. Mobile applications have become the primary channel through which citizens access information and services, making a mobile-first approach to SupplyCo stock information both technically feasible and socially impactful.

In this context, we present **MySupplyCo**, a comprehensive digital platform that bridges the information gap between SupplyCo outlets and the citizens they serve. The platform provides real-time inventory status information through an intuitive mobile application, enabling citizens to check stock availability, locate nearby outlets, and receive automated notifications when items are restocked — all without leaving their homes.

1.2 Problem Statement

The current system of information dissemination at Kerala SupplyCo outlets suffers from several critical deficiencies:

1. **Information Gap — Lack of Real-Time Visibility:** Citizens have no reliable mechanism to check the current stock levels at any SupplyCo outlet before making a physical visit. This creates uncertainty, frustration, and a pervasive sense of helplessness in accessing essential public services. The absence of real-time data means citizens are making purchasing decisions based on guesswork or outdated information.

2. **Wasted Citizen Effort:** Multiple trips to SupplyCo outlets are often necessary because citizens cannot verify stock availability remotely. Each unsuccessful visit wastes time (often 30–60 minutes including travel), transportation costs, and physical effort. For working-class citizens and daily-wage earners, these wasted trips represent a tangible economic loss.

3. **Manual and Unsustainable Communication:** In some areas, informal WhatsApp groups or landline telephone inquiries serve as ad-hoc information channels. However, these require SupplyCo staff to manually update stock information, diverting their attention from core responsibilities. This approach is neither scalable nor standardized, and the information shared is often incomplete or delayed.

4. **Digital Divide and Equity Concerns:** The lack of a digital platform for accessing public service information creates inequity in citizen experience. Tech-savvy individuals may find workarounds through social media or personal contacts, while elderly citizens and those with limited digital literacy are left entirely dependent on physical visits. This widens the gap between digitally connected and disconnected populations.

5. **Absence of Proactive Communication:** There exists no automated system to notify citizens when an out-of-stock item becomes available again. Citizens who are waiting for a particular commodity must repeatedly check with the outlet, resulting in multiple unnecessary visits and phone calls.

These problems collectively highlight the need for a digital solution that provides transparent, real-time, and accessible stock information to all citizens, regardless of their technical proficiency or physical mobility.

1.3 Objectives

The primary objectives of this project are:

1. **Real-Time Stock Visibility:** Develop a mobile application that displays current stock levels, pricing, and availability status for all commodities at any SupplyCo outlet in Kerala, updated through automated data synchronization.

2. **Reduce Citizen Effort:** Enable citizens to check stock availability from their mobile devices before travelling to an outlet, thereby eliminating unnecessary trips and reducing the time and cost associated with procurement of essential goods.

3. **Automated Stock Updates:** Implement an automated backend system that periodically synchronizes stock data from the government database (or its simulation) to the application's backend, ensuring data freshness without manual intervention.

4. **Proactive Restock Notifications:** Deliver real-time push notifications to registered users when previously out-of-stock items become available at their preferred outlet, enabling timely purchases.

5. **WhatsApp-Based Accessibility:** Provide a WhatsApp alert subscription interface as a complementary access channel, particularly designed for elderly users

and those who may find a dedicated application less accessible.

6. **Multi-Language Support:** Support eight Indian languages (English, Malayalam, Hindi, Gujarati, Marathi, Bengali, Tamil, and Telugu) to ensure the application is accessible to Kerala's diverse linguistic population, including migrant workers.

7. **Offline Capability:** Implement local caching mechanisms to ensure that users can view previously fetched stock data even when network connectivity is unavailable, with clear indicators of data freshness.

8. **Location-Based Store Discovery:** Integrate GPS-based geolocation services to help users discover SupplyCo outlets near their current location, complementing the search and filter functionality.

9. **Secure Architecture:** Design a system architecture that maintains strict data security through a read-only access model, ensuring no direct modification of government databases while still providing real-time information to citizens.

1.4 Novelty of Idea and Implementation Steps

1.4.1 Novelty

The MySupplyCo platform introduces several novel concepts in the domain of public distribution system digitization:

- **Three-Layer Read-Only Architecture:** Unlike traditional approaches that require direct integration with government systems, our architecture introduces a secure intermediary layer. The system comprises a Government Data Simulator (System A), a Main Backend Server (System B), and the Mobile Application. System B operates in a strictly read-only manner, fetching data from System A through authenticated API calls without any write access to government databases. This design ensures security while enabling real-time data delivery.

- **Incremental Checkpoint-Based Synchronization:** The data synchronization mechanism uses a checkpoint timestamp stored in a dedicated metadata table. During each sync cycle, only records modified since the last checkpoint are fetched, significantly reducing bandwidth consumption and server load compared to full-table synchronization approaches.

- **Database Webhook-Triggered Notifications:** Rather than polling for changes,

the system uses database webhooks that fire when stock quantities transition from zero to a positive value. This event-driven approach ensures that restock notifications are delivered to users within seconds of the actual restocking event.

- **Dual Access Channel Design:** The platform serves the same underlying data through two distinct channels — a feature-rich mobile application and a simplified WhatsApp alert interface — ensuring accessibility across different demographic segments and technology comfort levels.
- **Eight-Language Runtime Localization:** The application supports runtime language switching across eight Indian languages without requiring an app restart, with language preferences synchronized to the backend to enable language-appropriate push notification content in future iterations.

1.4.2 Implementation Steps

The project was implemented in the following systematic phases:

Step 1: **Requirements Analysis and Architecture Design:** Conducted feasibility studies, defined system requirements, and designed the three-layer architecture with clear interface boundaries.

Step 2: **Database Schema Design:** Designed the relational database schema comprising five tables (stores, items, stock, user_details, sync_metadata) with appropriate relationships and constraints.

Step 3: **Government Simulator Development:** Built the Django-based government data simulator with PostgreSQL, implementing REST API endpoints secured with API key authentication.

Step 4: **Main Backend Setup:** Configured Supabase as the main backend with PostgreSQL, set up authentication providers (OTP and Google OAuth), and defined Row Level Security policies.

Step 5: **Edge Function Development:** Developed two Supabase Edge Functions: `sync-stock` for periodic data synchronization and `send-push` for webhook-triggered push notifications.

Step 6: **Mobile Application Development:** Built the Flutter-based cross-platform mobile application with all UI screens, services layer, and offline caching capabilities.

Step 7: Localization Implementation: Implemented multi-language support using Flutter's ARB-based localization system with runtime language switching.

Step 8: Testing and Refinement: Conducted unit testing, integration testing, and user acceptance testing across multiple devices and network conditions.

1.5 Societal and Industrial Relevance

1.5.1 Alignment with Digital India

The MySupplyCo project directly aligns with the Government of India's Digital India programme, which envisions transforming India into a digitally empowered society and knowledge economy. By digitizing the stock information layer of the public distribution system, this project contributes to the programme's pillars of e-governance and digital empowerment of citizens. The Kerala state government's progressive stance on e-governance, exemplified by initiatives such as e-District and Akshaya centres, provides an ideal ecosystem for deploying such citizen-centric digital services.

1.5.2 Citizen Empowerment

By placing real-time information directly in the hands of citizens, MySupplyCo democratizes access to public service information. Citizens are no longer passive recipients dependent on physical visits or personal contacts; they become informed consumers who can make data-driven decisions about when and where to purchase essential commodities. This empowerment is particularly significant for marginalized communities who have historically had limited access to timely information.

1.5.3 Environmental Impact

The reduction in unnecessary trips to SupplyCo outlets has a direct positive environmental impact. Each avoided trip eliminates transportation-related carbon emissions, reduces traffic congestion, and decreases fuel consumption. Across 2000+ outlets serving millions of citizens, even a modest reduction in unnecessary visits translates to a substantial decrease in the collective carbon footprint of essential commodity procurement.

1.5.4 Scalable Framework for Government Services

The modular architecture designed for MySupplyCo serves as a reusable framework for digitizing other government services. The pattern of secure API-based data synchronization from government databases, combined with a citizen-facing mobile application, can be adapted for services ranging from ration card management to municipal service tracking. The read-only security model ensures that this framework can be deployed without requiring modifications to existing government IT infrastructure.

1.5.5 Industrial Relevance

From an industry perspective, this project demonstrates the practical application of modern software engineering practices including serverless computing (Supabase Edge Functions), event-driven architecture (database webhooks), cross-platform mobile development (Flutter), and Backend-as-a-Service (Supabase) patterns. These technologies represent the current state-of-the-art in application development and are widely adopted across the technology industry.

Chapter 2

Literature Review

2.1 Introduction

The digitization of public distribution systems and government supply chain operations has garnered significant attention in recent years, driven by the proliferation of mobile technology, cloud computing, and the growing emphasis on transparent governance. This chapter reviews existing literature, study reports, and comparable systems that inform the design and development of the MySupplyCo platform.

2.1.1 Study Reports

Several studies have examined the challenges and opportunities in digitizing India's Public Distribution System (PDS). The PDS, which serves approximately 800 million beneficiaries across India, is one of the largest food distribution networks in the world. Despite its scale, the system has been plagued by inefficiencies including leakage, diversion, and lack of transparency.

A comprehensive study by the National Institute of Public Finance and Policy (NIPFP) highlighted that technology-based interventions could reduce leakage in the PDS by up to 40% [2]. The study emphasized the importance of real-time monitoring and end-to-end digitization, recommending mobile-based solutions for both supply-side tracking and demand-side transparency.

The Ministry of Consumer Affairs, Food and Public Distribution has documented the implementation of the End-to-End Computerization of the Targeted Public Distribution System (TPDS) across Indian states. This initiative introduced Electronic Point

of Sale (EPOS) devices at fair price shops, enabling biometric authentication of beneficiaries and digital recording of transactions. However, the focus of EPOS remains on transaction authentication rather than consumer-facing stock information.

Research on mobile governance (m-governance) in India by Saxena (2020) demonstrated that mobile applications serve as the most effective channel for delivering government services to citizens, particularly in states with high smartphone penetration such as Kerala, where mobile internet usage exceeds 80% of the adult population [3]. The study concluded that citizen-facing applications must prioritize simplicity, multiple language support, and offline functionality to achieve broad adoption.

A study on supply chain visibility in public distribution by Rajan et al. (2021) specifically examined the Kerala context, noting that while the state's SupplyCo network is well-organized operationally, the information asymmetry between outlets and consumers remains a significant pain point. The study recommended real-time stock monitoring systems accessible via mobile applications as a priority intervention [4].

2.1.2 Existing Systems

Kerala SupplyCo Official Channels

The Kerala State Civil Supplies Corporation maintains an official website (`supplycokerala.com`) that provides general information about the organization, its outlets, and product listings. However, this portal does not offer real-time stock information at the individual outlet level. Citizens can access store addresses and contact numbers but cannot determine whether specific commodities are available before visiting [1].

Some SupplyCo outlets have adopted informal communication channels such as WhatsApp groups where staff manually share stock updates with local customers. While well-intentioned, this approach suffers from scalability limitations, inconsistent update frequency, and the burden it places on outlet staff who must manually compose and send messages.

EPOS and Aadhaar-Linked PDS

The Electronic Point of Sale (EPOS) system, mandated by the Government of India for fair price shops under the National Food Security Act, digitizes the transaction process at the point of sale. The system uses Aadhaar-based biometric authentication to verify beneficiary identity and records the quantity of commodities distributed. Several

states including Tamil Nadu, Andhra Pradesh, and Chhattisgarh have implemented state-specific PDS management applications.

However, these systems are primarily designed for supply-side management and government oversight. They record what was distributed and to whom, but do not provide a consumer-facing interface for checking what is currently available. The data flow is from the outlet to the government, not from the outlet to the citizen.

One Nation One Ration Card (ONORC)

The ONORC initiative enables beneficiaries to access their PDS entitlements from any fair price shop in India, regardless of their state of registration. While this system represents a significant step in PDS modernization, its focus is on portability of entitlements rather than stock visibility. The ONORC mobile application provides information about ration card details and transaction history but does not display real-time stock levels at individual outlets.

State-Level PDS Applications

Several Indian states have developed mobile applications for their PDS networks:

- **Tamil Nadu – TNPDS:** Provides ration card information, allotment details, and shop locator, but does not show real-time stock.
- **Andhra Pradesh – AP CFMS:** Focuses on financial management and transaction tracking for fair price shops.
- **Chhattisgarh – PDS Chhattisgarh:** Offers entitlement tracking and transaction history for beneficiaries.

None of these applications provide real-time, outlet-level stock availability information to consumers.

Private Sector Supply Chain Applications

In the private sector, applications such as BigBasket, JioMart, and Amazon Fresh have set user expectations for real-time inventory visibility in grocery retail. These platforms display current availability, pricing, and estimated delivery times, creating a contrast with the information opacity of government distribution systems. However, these commercial platforms operate on fundamentally different business models and have direct control over their inventory management systems.

International precedents include Singapore's FairPrice application and Japan's convenience store inventory systems, which provide real-time stock information to consumers. These systems demonstrate the technical feasibility and citizen value of inventory transparency in retail distribution.

2.1.3 Future Directions and Research Opportunities

The literature suggests several emerging technologies that could further enhance public distribution system digitization:

- **AI-Based Demand Prediction:** Machine learning models trained on historical consumption patterns could predict stock depletion rates and trigger preemptive restocking, reducing the frequency and duration of out-of-stock events.
- **Blockchain for Supply Chain Transparency:** Distributed ledger technology could provide an immutable record of commodity movement from procurement to distribution, enhancing accountability and reducing diversion.
- **IoT-Based Automatic Stock Sensing:** Internet of Things sensors (weight scales, RFID tags) could enable automatic stock level detection, eliminating the need for manual data entry and ensuring continuous real-time accuracy.
- **Conversational AI and Chatbots:** Natural language processing-based chatbots on platforms like WhatsApp could enable voice-based stock queries, further lowering the digital literacy barrier for elderly and rural users.

2.2 Conclusions and Gap Analysis

2.2.1 Conclusion

The literature review reveals that while significant progress has been made in digitizing the supply side of India's Public Distribution System through initiatives like EPOS, ONORC, and state-level management applications, the demand side — specifically, consumer-facing stock visibility — remains largely unaddressed. Existing systems focus on transaction authentication, entitlement management, and administrative oversight, but do not empower citizens with the real-time information needed to make informed decisions about when and where to purchase essential commodities.

The private sector has demonstrated that real-time inventory visibility is technically achievable, economically viable, and highly valued by consumers. The gap lies in

applying these principles to government distribution systems where direct database access is restricted and security concerns are paramount.

2.2.2 Gap Analysis

Based on the review, the following gaps have been identified that the MySupplyCo project addresses:

1. **No Real-Time Consumer-Facing Stock Information:** None of the existing government PDS applications provide outlet-level, real-time stock availability information to citizens. MySupplyCo fills this gap with automated data synchronization and a user-friendly mobile interface.

2. **No Proactive Notification System:** Existing systems do not notify citizens when out-of-stock commodities become available. MySupplyCo implements database webhook-triggered push notifications for restock events.

3. **Limited Language Support:** Most government applications are available only in English and one regional language. MySupplyCo supports eight Indian languages with runtime switching capability.

4. **No Offline Capability:** Existing PDS applications require active internet connectivity. MySupplyCo implements local caching to provide offline access to previously fetched stock data.

5. **No Location-Based Discovery:** Current systems require users to know the name or address of their desired outlet. MySupplyCo integrates GPS-based nearby store discovery.

6. **No Accessible Alternative Channel:** Existing applications offer only app-based access. MySupplyCo provides a WhatsApp alert subscription as a complementary channel for digitally less-comfortable users.

7. **No Secure Read-Only Architecture for Government Data:** Most approaches either require direct database access or operate with stale data. MySupplyCo's three-layer architecture provides near-real-time data access through a secure, intermediary synchronization layer.

2.3 Summary

This literature review establishes that while India's PDS has undergone significant digitization on the administrative side, the consumer-facing dimension remains an underserved area. The MySupplyCo project addresses this gap by providing a secure, scalable, and accessible platform for real-time stock visibility. The three-layer architecture, combined with multi-language support, offline capability, and proactive notifications, represents a novel approach to citizen-centric public distribution system digitization. The system draws on lessons from both government e-governance initiatives and private sector inventory management practices while addressing the unique security and scalability constraints of government service delivery.

Chapter 3

Feasibility Study and Requirements Analysis

3.1 Feasibility

A comprehensive feasibility analysis was conducted to evaluate the viability of the MySupplyCo project across multiple dimensions. This analysis ensures that the proposed system is not only technically achievable but also economically sustainable, legally compliant, and operationally practical within the constraints of a government service delivery context.

3.1.1 Technical Feasibility

The technical feasibility of the project is assessed based on the availability and maturity of the technologies required for implementation.

- **Flutter Framework:** Flutter, developed by Google, is a mature open-source UI software development kit for building cross-platform applications from a single codebase. It compiles to native ARM code for both Android and iOS, ensuring high performance. Flutter's widget-based architecture and hot-reload capability make it suitable for rapid prototyping and iterative development. The framework has been adopted by major organizations including Google, Alibaba, and BMW, demonstrating its production readiness [5].
- **Supabase (Backend-as-a-Service):** Supabase is an open-source alternative to Firebase, built on PostgreSQL. It provides authentication, real-time subscrip-

tions, storage, and serverless edge functions out of the box. Its open-source nature ensures no vendor lock-in, and its PostgreSQL foundation provides enterprise-grade reliability and SQL compatibility. Supabase's Row Level Security (RLS) policies enable fine-grained access control at the database level [6].

- **Django Framework:** Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. For the government data simulator, Django's ORM (Object-Relational Mapping), built-in admin interface, and REST framework extension make it ideal for quickly building and managing a mock government database with API endpoints.
- **PostgreSQL:** PostgreSQL is the world's most advanced open-source relational database system. It supports advanced features including JSON data types, full-text search, geospatial queries (via PostGIS), and stored procedures. Both the government simulator and the main backend use PostgreSQL, ensuring compatibility and leveraging its robust indexing and query optimization capabilities.
- **Firebase Cloud Messaging (FCM):** FCM provides a reliable and battery-efficient connection between the server and devices for delivering push notifications at no cost. It supports both Android and iOS platforms and handles message queuing, delivery confirmation, and topic-based messaging [7].
- **Geolocator:** The Geolocator package for Flutter provides cross-platform access to GPS location services with configurable accuracy levels. Combined with PostGIS functions on the server side, this enables efficient nearby store discovery based on the user's current geographical coordinates.

All technologies selected are mature, well-documented, and actively maintained by large communities. The project team has prior experience with Flutter and web development, making the technical implementation feasible within the project timeline.

3.1.2 Economic Feasibility

The economic feasibility of the project is highly favourable due to the exclusive use of open-source technologies and free-tier cloud services:

Table 3.1: Cost Analysis of Project Components

Component	**Service/Tool**	**Cost**
Backend Database	Supabase Free Tier (500 MB database, 50,000 monthly active users)	Free
Push Notifications	Firebase Cloud Messaging (unlimited messages)	Free
Authentication	Supabase Auth (50,000 monthly active users)	Free
Edge Functions	Supabase Edge Functions (500,000 invocations/month)	Free
Government Simulator	Django on local/university server	Free
Development Tools	Flutter SDK, VS Code, Android Studio, Git	Free
Testing Devices	Team members' personal Android devices	No cost
Domain/Hosting	Not required for prototype phase	Free

The total development cost for the prototype phase is effectively zero, as all tools, frameworks, and hosting services operate within their free tiers. For a production deployment, Supabase's Pro plan ($25/month) and a Django hosting server ($5–10/month) would represent the only recurring costs, making the system economically viable even for government budget constraints.

3.1.3 Time Feasibility

The project was planned for execution within the S6 semester (approximately 6 months), divided into the following phases:

Table 3.2: Project Timeline

Phase	**Activities**	**Duration**
Phase 1	Requirements analysis, architecture design, UI/UX prototyping	4 weeks
Phase 2	Database schema design, Django simulator development	3 weeks
Phase 3	Supabase backend setup, edge function development	3 weeks
Phase 4	Flutter app development (core screens)	5 weeks
Phase 5	Localization, notifications, offline caching	3 weeks
Phase 6	Testing, bug fixing, documentation	4 weeks

The timeline is feasible given the team size of three developers and the modular nature

of the architecture, which allows parallel development of the frontend, backend, and simulator components.

3.1.4 Legal Feasibility

The legal aspects of the project have been carefully considered:

- **No Government System Modification:** The system operates in a strictly read-only mode. It does not modify, delete, or write any data to government databases. The government simulator is an independent system created solely for testing purposes.
- **Data Privacy Compliance:** The application collects minimal personal data (name, phone number, optional email) required for authentication and notification delivery. A comprehensive privacy policy has been implemented within the application, clearly stating data collection practices, usage limitations, and user rights.
- **Open-Source Licensing:** All technologies used (Flutter, Supabase, Django, PostgreSQL, Firebase) are available under permissive open-source licenses (BSD, Apache 2.0, MIT) that allow commercial and government use without licensing fees or legal restrictions.
- **Terms and Conditions:** The application includes terms and conditions that define user responsibilities, acceptable use policies, and limitation of liability, ensuring legal protection for both the service provider and users.

3.1.5 Operational Feasibility

The operational feasibility assessment evaluates whether the system can be effectively deployed and maintained in a real-world government setting:

- **Modular Architecture for Government Integration:** The three-layer architecture is designed so that the government simulator (System A) can be replaced with the actual government API when it becomes available. Only the synchronization layer needs modification; the mobile application and main backend remain unchanged.
- **Minimal Training Requirements:** The mobile application is designed with an intuitive user interface following Material Design guidelines. The onboarding flow, guest mode access, and multi-language support ensure that users with

varying levels of digital literacy can use the application with minimal or no training.

- **Scalable Infrastructure:** Supabase's cloud infrastructure can scale horizontally to handle increased load as more outlets and users are onboarded. The incremental synchronization strategy ensures that the backend load grows linearly with the number of modified records, not the total dataset size.
- **Low Maintenance Overhead:** The serverless edge functions require no server management. Supabase handles database backups, security patches, and infrastructure updates. The only operational requirement is monitoring the sync function's execution and addressing any API connectivity issues.

3.2 Project Requirements

3.2.1 Implementation Requirements

Hardware Requirements

Table 3.3: Hardware Requirements

Component	**Specification**
Development Machine	Intel Core i5 or equivalent, 8 GB RAM, 256 GB SSD
Testing Device (Android)	Android 8.0+, 2 GB RAM, GPS-enabled
Testing Device (iOS)	iPhone with iOS 14.0+ (optional)
Internet Connection	Broadband with minimum 10 Mbps for development

Software Requirements

Table 3.4: Software Requirements

Category	Software	Version
Programming Language	Dart	3.10.4+
Mobile Framework	Flutter SDK	3.x
Backend Platform	Supabase	Latest
Server Framework	Django	4.x
Database	PostgreSQL	15+
Push Notifications	Firebase Cloud Messaging	Latest
IDE	Visual Studio Code / Android Studio	Latest
Version Control	Git, GitHub	Latest
Design Tool	Figma	Web version
Operating System	Windows 10/11, macOS, or Linux	Latest

3.2.2 Deployment Requirements

For the prototype phase, the following deployment setup is required:

- **Supabase Cloud:** A Supabase project instance for hosting the PostgreSQL database, authentication service, and edge functions. The free tier is sufficient for prototype deployment.
- **Django Server:** A Python-capable server (local machine or cloud VM) for hosting the government data simulator. For production deployment, this would be replaced by the actual government API endpoint.
- **Firebase Project:** A Google Firebase project configured with Cloud Messaging for push notification delivery. A service account JSON key is required for server-side FCM authentication.
- **Application Distribution:** For testing, APK files are distributed directly. For production, the application would be published on the Google Play Store and optionally the Apple App Store.
- **Environment Configuration:** A `.env` file containing Supabase URL, Supabase anonymous key, and other configuration parameters must be maintained for the Flutter application.

Chapter 4

Project Design

4.1 Introduction

4.1.1 Purpose

The purpose of this chapter is to present the detailed design of the MySupplyCo system, including its architecture, database schema, data flow, and user interaction patterns. The design aims to create a secure, scalable, and user-friendly platform that provides real-time stock information from Kerala SupplyCo outlets to citizens through a mobile application.

4.1.2 Scope

The system design encompasses three interconnected components:

1. A government data simulator that mimics the central SupplyCo database and exposes stock data through a secure REST API.
2. A main backend server that synchronizes data from the government source, manages user authentication, and delivers push notifications.
3. A cross-platform mobile application that presents stock information to users with offline capability, multi-language support, and location-based store discovery.

4.1.3 Overview

The MySupplyCo system follows a three-layer architecture where each layer has clearly defined responsibilities and communicates with adjacent layers through well-defined

interfaces. This separation of concerns ensures that changes to one layer (such as replacing the government simulator with an actual API) do not require modifications to other layers.

4.2 System Architecture

4.2.1 Software Design

The software design is presented through multiple diagram types that capture different aspects of the system: the overall architecture, entity relationships, use cases, and data flow.

System Architecture Diagram

Figure 4.1 illustrates the three-layer architecture of the MySupplyCo system, showing the components within each layer and the communication protocols between them.

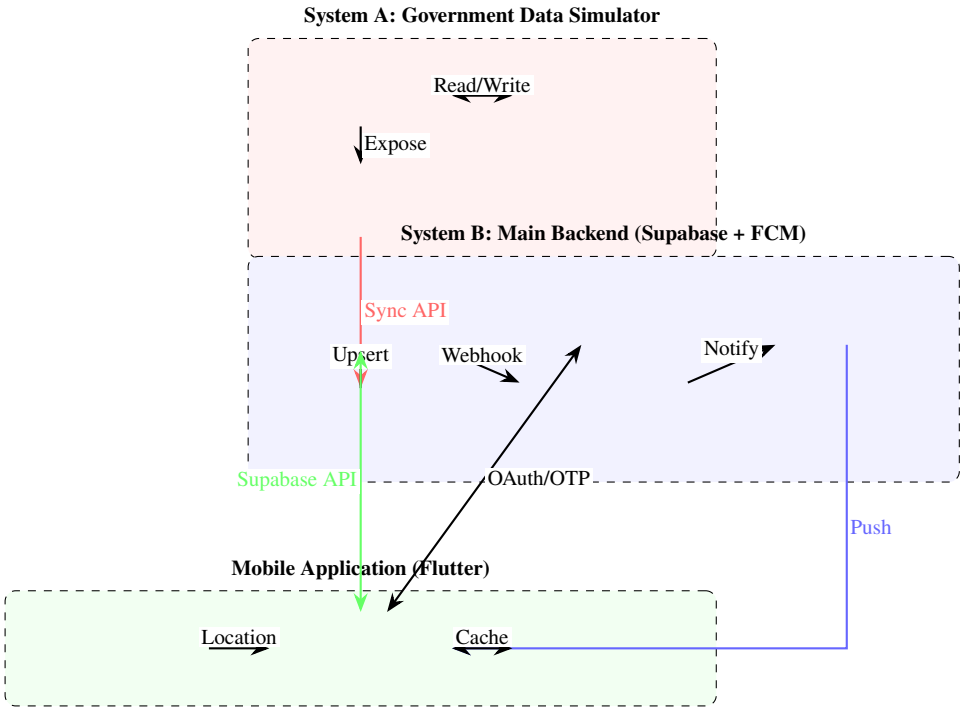

Figure 4.1: System Architecture Diagram of MySupplyCo

Entity-Relationship Diagram

Figure **??** presents the Entity-Relationship diagram showing the seven database tables and their relationships.

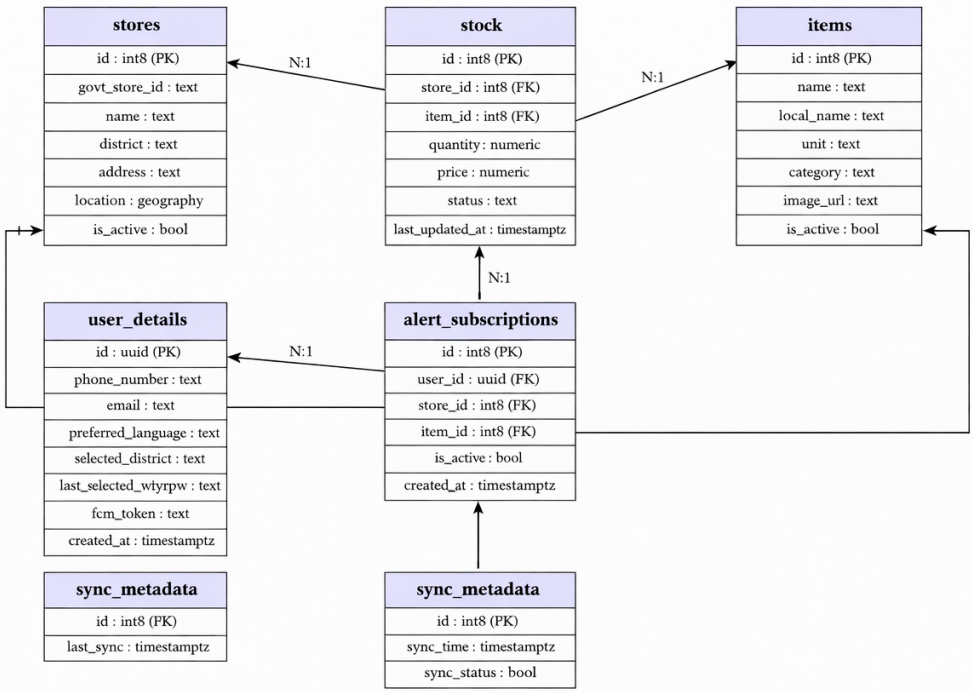

Figure 4.2: Entity-Relationship Diagram

The database schema consists of the following tables:

- **stores:** Contains information about each SupplyCo outlet including name, district, address, geographical location, and the government-assigned store identifier.
- **items:** Master table of all commodities available across the SupplyCo network, with name, local name, unit of measurement, image URL, and category.
- **stock:** Junction table that maps stores to items with current quantity, price, and availability status. Uses a composite unique constraint on (store_id, item_id) for upsert operations during synchronization.
- **user_details:** Stores registered user information including authentication details, preferred district, last visited store, FCM token for push notifications, and language preference.
- **alert_subscriptions:** Tracks user subscriptions for restock alerts on specific items at specific stores, enabling targeted push notifications.

- **sync_metadata:** Maintains the last synchronization timestamp used for incremental data fetching from the government simulator.
- **log_data:** Records synchronization events with timestamp and status for monitoring and debugging purposes.

Use Case Diagram

Figure 4.2 shows the use case diagram depicting the interactions between system actors and the major functionalities.

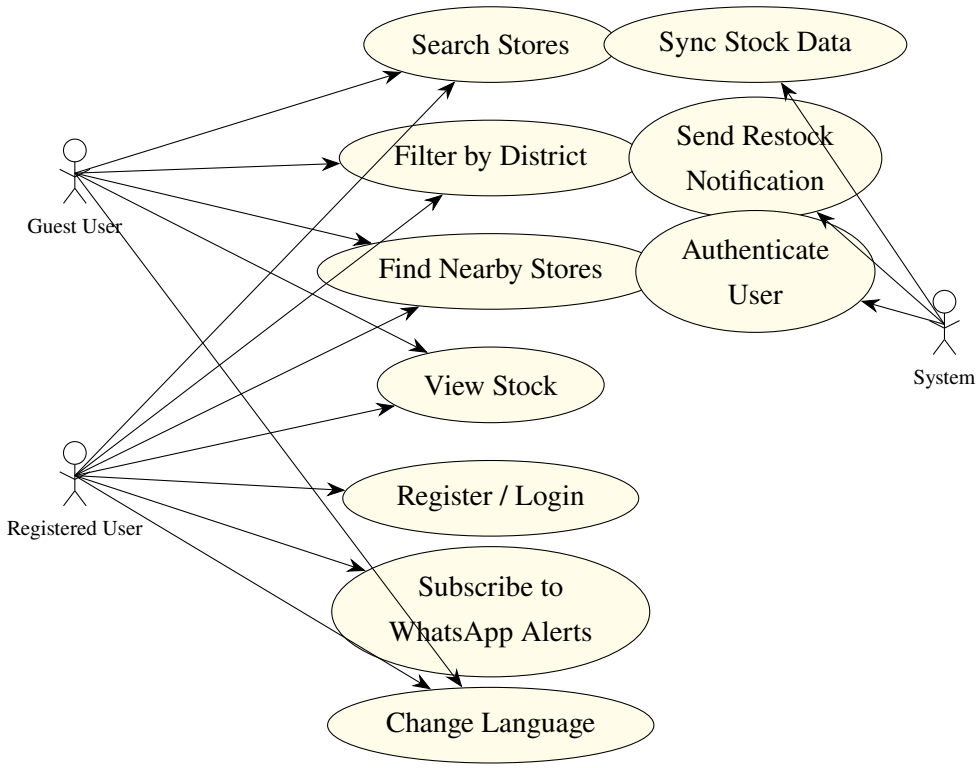

Figure 4.2: Use Case Diagram

Data Flow Diagram

Figure 4.3 illustrates the data flow through the system, showing how stock data moves from the government source to the end user.

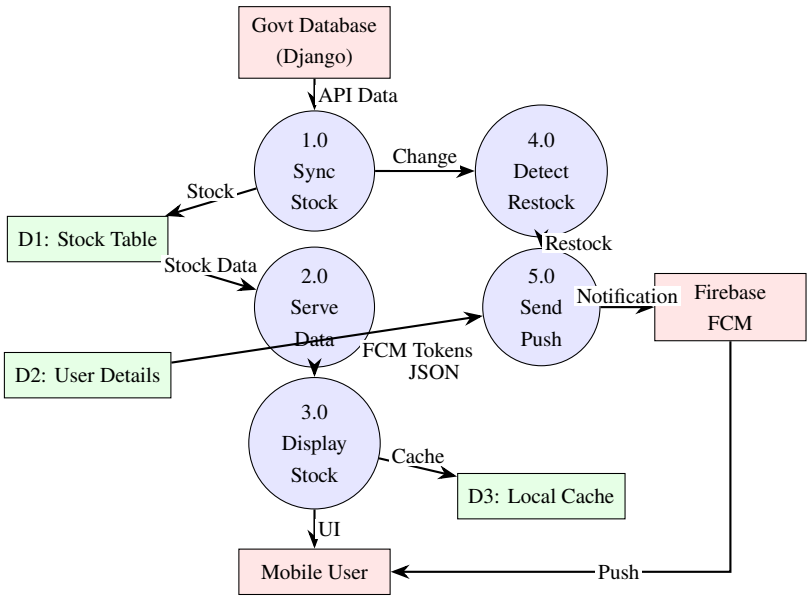

Figure 4.3: Data Flow Diagram

Application Navigation Flow

Figure 4.4 presents the screen navigation flow of the mobile application, showing the possible transitions between different screens based on user state (first-time, returning, guest, or authenticated).

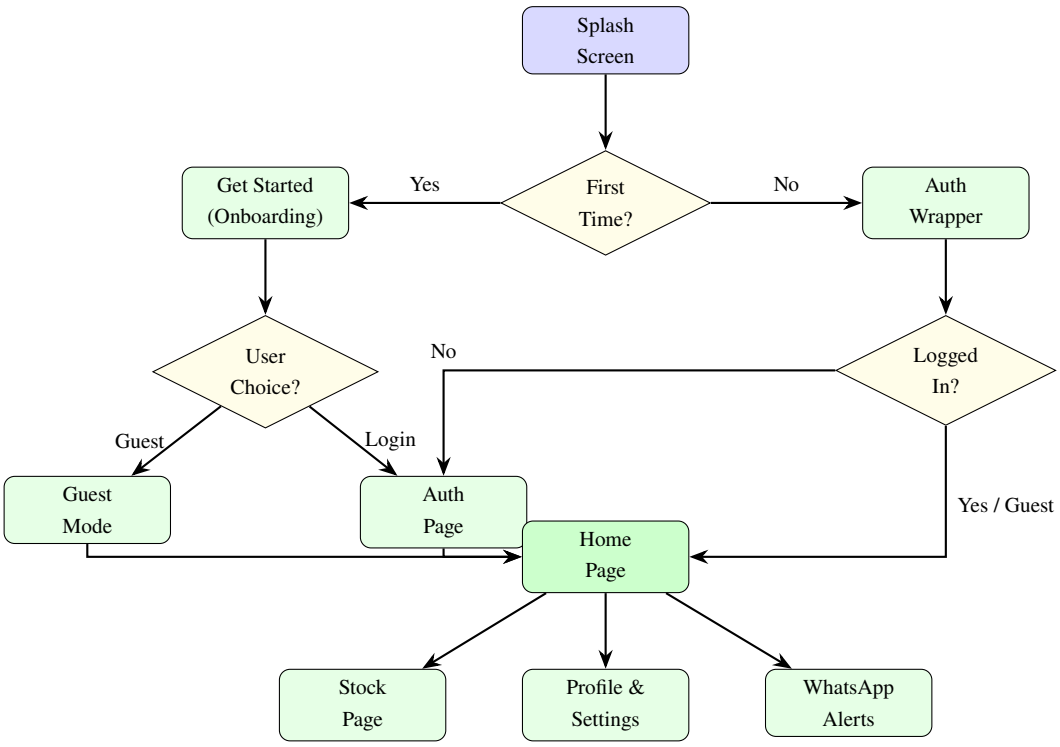

Figure 4.4: Application Navigation Flow

4.2.2 Architectural Overview

The following subsections provide a detailed description of each architectural layer and its internal components.

System A: Government Data Simulator

The government data simulator serves as a prototype representation of the central SupplyCo database infrastructure. In a real-world deployment, this would be replaced by the actual government API.

Technology Stack:

- **Framework:** Django 4.x with Django REST Framework
- **Database:** PostgreSQL 15+
- **Authentication:** API Key-based authentication via custom HTTP headers (`X-API-KEY`)

API Endpoint: The simulator exposes a single REST endpoint that accepts a `last_sync_time` query parameter and returns all stock records modified after that

timestamp. This enables incremental synchronization, where only changed records are transferred during each sync cycle.

Data Model: The simulator maintains its own set of stores, items, and stock records, simulating the data structure of the real SupplyCo central database. Stock quantities and prices are periodically updated to simulate real-world inventory changes.

System B: Main Backend Server (Supabase)

The main backend serves as the intermediary between the government data source and the mobile application. It is built on Supabase, an open-source Backend-as-a-Service platform.

Components:

1. **PostgreSQL Database:** Hosts five tables (stores, items, stock, user_details, sync_metadata) as described in the ER diagram. Row Level Security (RLS) policies ensure that users can only access their own profile data while stock and store data is publicly readable.
2. **Supabase Auth:** Manages user authentication through two providers:
   - **Phone OTP:** Users receive a one-time password via SMS to their registered mobile number.
   - **Google OAuth:** Users can sign in using their Google account through OAuth 2.0 flow with deep link callback (`supplyco://login-callback/`).
3. **Edge Function — sync-stock:** A serverless Deno function that:
   - Reads the last sync timestamp from the `sync_metadata` table.
   - Calls the Django API with the timestamp to fetch only modified records.
   - Upserts the received data into the `stock` table using the composite key (store_id, item_id).
   - Updates the sync checkpoint timestamp.
4. **Edge Function — send-push:** A serverless Deno function triggered by a database webhook when the `stock` table is updated. It:
   - Compares old and new stock quantities to detect restock events (quantity changing from 0 to a positive value).

- Queries the item name, store name, and affected user FCM tokens.
- Authenticates with Firebase using a service account JWT.
- Sends personalized push notifications via the FCM v1 API.

5. **RPC Function — get_nearby_stores:** A PostgreSQL stored procedure that accepts latitude and longitude parameters and returns stores ordered by geographical distance, leveraging PostGIS spatial functions.

Mobile Application (Flutter)

The mobile application is the citizen-facing component of the system, built using the Flutter framework for cross-platform compatibility.

Architecture Pattern: The application follows a service-oriented architecture with a clear separation between UI components and business logic:

- **UI Layer:** Composed of screen widgets organized into three categories:
  - `intro_pages/` — Splash screen, onboarding, and authentication screens.
  - `core_pages/` — Home page (store listing) and stock page (item inventory view).
  - `other_pages/` — Settings, alerts, help, privacy policy, and terms pages.
- **Services Layer:** Five service classes that encapsulate all business logic:
  - `SupabaseService` — Centralized data access layer for all database operations (fetch stores, fetch stock, manage user details, nearby store queries).
  - `NotificationService` — Manages FCM token registration, permission requests, and token refresh handling.
  - `LocationService` — Handles GPS permission management and current position retrieval.
  - `StorageService` — Local persistence via SharedPreferences including offline stock caching, onboarding state, guest mode, and language preferences.
  - `LocalizationProvider` — ChangeNotifier-based provider for reactive runtime language switching across eight supported locales.

- **Localization Layer:** Application Resource Bundle (ARB) files define translated strings for all eight languages. Flutter's code generation system produces Dart classes for type-safe access to localized strings.

Key Design Decisions:

1. **Guest Mode:** The application allows full read-only access without authentication, lowering the barrier to entry. Registered users gain access to push notifications and WhatsApp alert subscriptions.
2. **Offline-First Stock Display:** When opening a previously visited store, cached stock data is displayed immediately while a fresh fetch occurs in the background. If the network request fails, the cached data remains visible with a clear "Offline" banner and timestamp.
3. **Provider Pattern for State Management:** The Provider package is used for dependency injection and state management, following Flutter's recommended approach for applications of this scale.

Chapter 5

Implementation and Testing

5.1 Functionality Implementation and Test Cases

This section details the implementation of each major functionality of the MySupplyCo application, along with the corresponding test cases used to validate correct behaviour.

5.1.1 User Authentication Functionality

Input:

- User's full name (registration only).
- Mobile phone number (10 digits, Indian format with +91 prefix).
- One-Time Password (OTP) received via SMS.
- Alternatively, Google account credentials for OAuth sign-in.

Output:

- Successful registration creates a new entry in the `user_details` table.
- Successful login establishes an authenticated session and redirects to the Home Page.
- Guest mode allows unauthenticated read-only access to stock information.

Process:

1. **Registration Flow:** User enters full name and mobile number on the Register tab. The system sends an OTP via Supabase Auth (`signInWithOtp`). User

enters the received OTP. The system verifies the OTP (`verifyOTP`) and creates a user profile in the `user_details` table via upsert. The user's preferred language is synced to the backend.

2. **Login Flow:** User enters their registered mobile number on the Login tab. OTP is sent and verified. The system checks if a profile exists in `user_details`. If found, the user is redirected to the Home Page. If not found, the user is prompted to register first.

3. **Google OAuth Flow:** User taps "Continue with Google". The system initiates OAuth via `signInWithOAuth` with a deep link callback (`supplyco://login-callback/`). On the Register tab, a new profile is created with the entered name and Google email. On the Login tab, existing profiles are verified before granting access.

4. **Guest Mode:** User taps "Continue as Guest" on the onboarding screen. The system sets a local flag via `StorageService.setGuestMode(true)` and grants read-only access to stock viewing features without authentication.

Test Cases:

1. **Input:** Valid name and 10-digit phone number, correct OTP. **Expected Output:** User is registered, profile saved to database, redirected to Home Page.

2. **Input:** Phone number with fewer than 10 digits. **Expected Output:** Error message "Please enter a valid 10-digit mobile number" displayed.

3. **Input:** Correct phone number but incorrect OTP. **Expected Output:** Authentication error message displayed, user remains on Auth Page.

4. **Input:** Login attempt with unregistered phone number. **Expected Output:** Message "Number not registered" displayed, user redirected to Register tab.

5. **Input:** Google Sign-In on Register tab without entering name. **Expected Output:** Error message "Please enter your name before continuing with Google".

6. **Input:** Guest mode selection. **Expected Output:** User can view stores and stock but cannot access notification subscriptions.

5.1.2 Store Discovery Functionality

Input:

- Search query (store name or partial name).
- District selection from dropdown filter.
- GPS coordinates (for nearby store discovery).

Output:

- List of matching SupplyCo stores with name, district, and address.
- Nearby stores ordered by geographical distance.

Process:

1. **Search:** User types a store name in the search bar. The system performs client-side filtering on the cached store list, showing real-time suggestions as the user types. Pressing the search button or selecting "View all" navigates to a full results page.
2. **District Filter:** User selects a district from the dropdown menu populated with all Kerala districts. The store list is filtered to show only outlets in the selected district.
3. **Nearby Stores:** The system requests GPS permissions via `LocationService`. On permission grant, the current position is obtained via `Geolocator.getCurrentPosition()` with high accuracy. The coordinates are sent to the `get_nearby_stores` RPC function, which returns stores ordered by distance.

Test Cases:

1. **Input:** Search query "Thrissur". **Expected Output:** All stores with "Thrissur" in their name or address are displayed.
2. **Input:** District filter set to "Ernakulam". **Expected Output:** Only stores in Ernakulam district are shown.
3. **Input:** GPS location permission granted. **Expected Output:** Nearby stores displayed with distance information, ordered from nearest to farthest.
4. **Input:** GPS permission denied. **Expected Output:** Appropriate error message displayed, manual search remains functional.
5. **Input:** Empty search query. **Expected Output:** All stores displayed (no filter applied).

5.1.3 Stock Viewing Functionality

Input:

- Store selection (tap on a store from the list).
- Optional search query within stock items.

Output:

- Grid view of all stock items with name, quantity, unit, price, item image, and availability status.
- Statistics chips showing total items and available items count.
- Special offers tab with promotional information.

Process:

1. User taps on a store from the Home Page. The Stock Page opens with two tabs: "Stock Items" and "Special".
2. If cached stock exists for the selected store, it is displayed immediately while a fresh fetch occurs in the background.
3. The system calls `SupabaseService.fetchStock(storeId)`, which queries the `stock` table with a join to the `items` table.
4. Fetched data is displayed in a 2-column grid of stock cards, each showing the item image, name, quantity with unit, price in INR, and an availability badge (green for available, red for unavailable).
5. Successfully fetched data is saved to local cache via `StorageService.saveStoreSnapshot()`.
6. If the network request fails, cached data is shown with an "Offline" banner indicating the data age.

Test Cases:

1. **Input:** Store with 15 stock items selected. **Expected Output:** All 15 items displayed in grid view with correct details.
2. **Input:** Search query "rice" within stock items. **Expected Output:** Only items containing "rice" in their name are displayed.

3. **Input:** Store selected while device is offline, with cached data. **Expected Output:** Cached stock displayed with offline banner showing time since last update.
4. **Input:** Store selected while device is offline, no cached data. **Expected Output:** Error message with empty state placeholder displayed.
5. **Input:** Refresh button tapped while online. **Expected Output:** Stock data refreshed from server, cache updated.

5.1.4 Push Notification System

Input:

- Database webhook trigger when stock table is updated.
- User's FCM token stored in `user_details`.

Output:

- Push notification on user's device: "Restock at [Store Name]! [Item Name] is now back in stock."

Process:

1. On app launch, `NotificationService` requests notification permissions from the user.
2. If authorized, the FCM token is retrieved and saved to the `user_details` table in Supabase.
3. A token refresh listener ensures the token is updated if it changes.
4. When the `sync-stock` edge function updates the stock table, the database webhook triggers the `send-push` edge function.
5. The `send-push` function compares the old and new quantity values. If the old quantity was 0 and the new quantity is greater than 0 (restock event), it proceeds.
6. The function queries for users whose `last_selected_supplyco` matches the store and who have a valid FCM token.
7. It authenticates with Firebase using a service account JWT and sends personalized notifications via the FCM v1 API.

Test Cases:

1. **Input:** Stock item quantity changes from 0 to 50. **Expected Output:** Push notification sent to all users subscribed to that store.
2. **Input:** Stock quantity changes from 20 to 50. **Expected Output:** No notification sent (not a restock from zero).
3. **Input:** User denies notification permission. **Expected Output:** No FCM token saved, user does not receive notifications but app remains functional.
4. **Input:** User's FCM token expires and is refreshed. **Expected Output:** New token automatically saved to Supabase via the `onTokenRefresh` listener.

5.1.5 WhatsApp Alert Subscription

Input:

- User's WhatsApp phone number (10 digits).
- Selection of commodities to monitor (from a list of 13 items).
- Optional "Notify me about special offers" checkbox.

Output:

- Subscription confirmation displayed to the user.
- User preferences saved for future WhatsApp alert delivery.

Process:

1. User navigates to the WhatsApp Alerts page from the Home Page.
2. User enters their WhatsApp number and selects commodities from the checklist (Rice, Sugar, Coconut Oil, Wheat, Dal variants, Mustard Oil, Salt, Onion, Potato, Tomato, Milk, Atta).
3. A "Select All / Deselect All" toggle is available for convenience.
4. User taps "Enable WhatsApp Alerts" button.
5. The system validates the phone number (exactly 10 digits, numeric only) and ensures at least one commodity or special offers is selected.
6. On successful validation, a confirmation is displayed and the user is redirected to the Home Page.

Test Cases:

1. **Input:** Valid 10-digit number, 3 commodities selected. **Expected Output:** Subscription confirmed, user redirected to Home Page.
2. **Input:** Phone number with 8 digits. **Expected Output:** Error message "Please enter a valid 10-digit phone number".
3. **Input:** Valid phone number, no commodities selected, special offers unchecked. **Expected Output:** Error message "Please select at least one item or special offers".
4. **Input:** "Select All" toggled on. **Expected Output:** All 13 commodity checkboxes become checked.

5.1.6 Localization (Multi-Language Support)

Input:

- Language selection from the language dialog or onboarding screen.

Output:

- Entire application UI dynamically updates to the selected language without requiring a restart.

Process:

1. On first launch, the onboarding screen presents language options: English (default) and "Others" (expanding to show Malayalam, Hindi, Gujarati, Marathi, Bengali, Tamil, Telugu).
2. Language selection updates `LocalizationProvider`, which calls `notifyListeners()` to trigger a reactive UI rebuild.
3. The selected language code is persisted locally via `StorageService.setPreferredLanguage(code)`.
4. For authenticated users, the language preference is synced to the `user_details.preferred_language` column in Supabase.
5. The `MaterialApp` widget consumes the locale from the provider and loads the corresponding ARB-generated localization delegates.

Test Cases:

1. **Input:** Language changed from English to Malayalam. **Expected Output:** All UI strings (buttons, labels, titles, descriptions) display in Malayalam.
2. **Input:** Language set to Hindi, app restarted. **Expected Output:** App launches in Hindi (persisted preference loaded from local storage).
3. **Input:** Authenticated user changes language. **Expected Output:** Language preference synced to Supabase `user_details` table.

5.1.7 Profile and Settings

Input:

- Navigation to Profile & Settings page.
- User interactions: language change, logout, navigation to sub-pages.

Output:

- User profile information displayed (name, masked phone number).
- Access to settings: language, accessibility, help, terms, privacy policy.
- Logout functionality clearing the session.

Process:

1. On page load, `_loadUserDetails()` fetches the user's name, phone number, and language preference from the `user_details` table.
2. The profile card displays the user's name in uppercase with a circular avatar (first letter of name) and a masked phone number (e.g., `+91******4321`).
3. Guest users see "GUEST USER" with no phone number.
4. Settings are organized into three groups: Profile/Alerts, Language/Accessibility, and Support/Legal.
5. The logout button calls `supabase.auth.signOut()` and navigates to the Auth Page, clearing the navigation stack.

Test Cases:

1. **Input:** Authenticated user opens Profile page. **Expected Output:** User's name and masked phone number displayed correctly.

2. **Input:** Guest user opens Profile page. **Expected Output:** "GUEST USER" displayed, Set Profile option available.
3. **Input:** User taps "Log Out". **Expected Output:** Session cleared, user redirected to Auth Page.

5.2 Testing Strategies

The following testing strategies were employed throughout the development of the MySupplyCo application:

1. **Unit Testing:**
   - Writing tests for individual service methods (e.g., `SupabaseService.fetchStock()`, `StorageService.getTimeAgo()`) to verify correct data transformation and error handling in isolation.
   - Testing localization helper methods for correct language code mapping.
2. **Integration Testing:**
   - Testing the interaction between the Flutter application and Supabase backend, verifying authentication flows, data fetching, and profile management end-to-end.
   - Verifying the sync-stock edge function correctly fetches data from the Django API and upserts it into the Supabase database.
   - Testing the send-push edge function's restock detection logic and FCM notification delivery.
3. **End-to-End Testing:**
   - Simulating complete user journeys: from app launch through onboarding, store selection, stock viewing, and notification receipt.
   - Testing the offline-to-online transition: verifying cached data display, background refresh, and cache update.
4. **User Interface Testing:**
   - Ensuring UI consistency and responsiveness across different Android screen sizes and densities.

- Verifying that all eight language translations render correctly without text overflow or layout issues.
- Testing accessibility features including color contrast and touch target sizes.

5. **Location-Based Testing:**
   - Testing GPS permission flows (granted, denied, permanently denied) on physical devices.
   - Verifying the accuracy of nearby store results using known GPS coordinates.
   - Testing behaviour when location services are disabled at the device level.

5.3 Source Code

The complete source code for the MySupplyCo project is available on GitHub:

https://github.com/Abhinand-Ullas/Supply_co_app

The repository contains:

- Flutter mobile application source code (`lib/` directory)
- Supabase Edge Functions (`lib/supabase_backend/`)
- Localization files for 8 languages (`lib/generated_localizations/`)
- Asset files including app logo and launcher images (`lib/images/`)
- Configuration files (`pubspec.yaml`, `l10n.yaml`, `analysis_options.yaml`)

5.4 Screenshots

The following screenshots demonstrate the key screens of the MySupplyCo mobile application.

Figure 5.1: Splash Screen

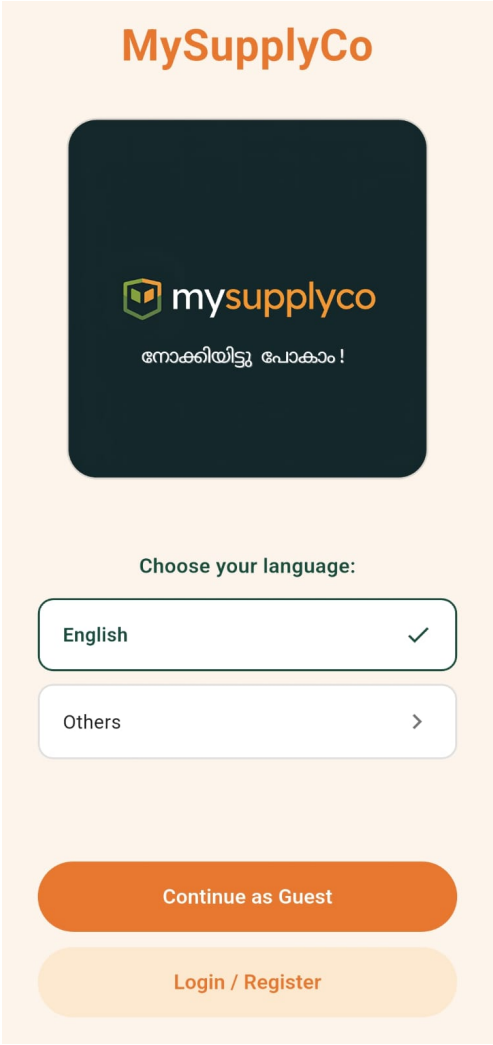

Figure 5.2: Get Started / Onboarding Screen

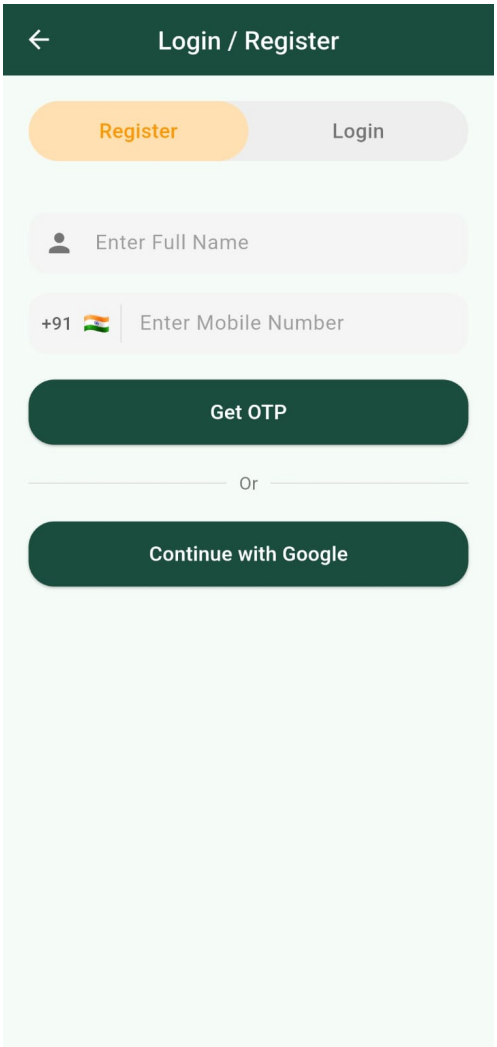

Figure 5.3: Login / Register Screen

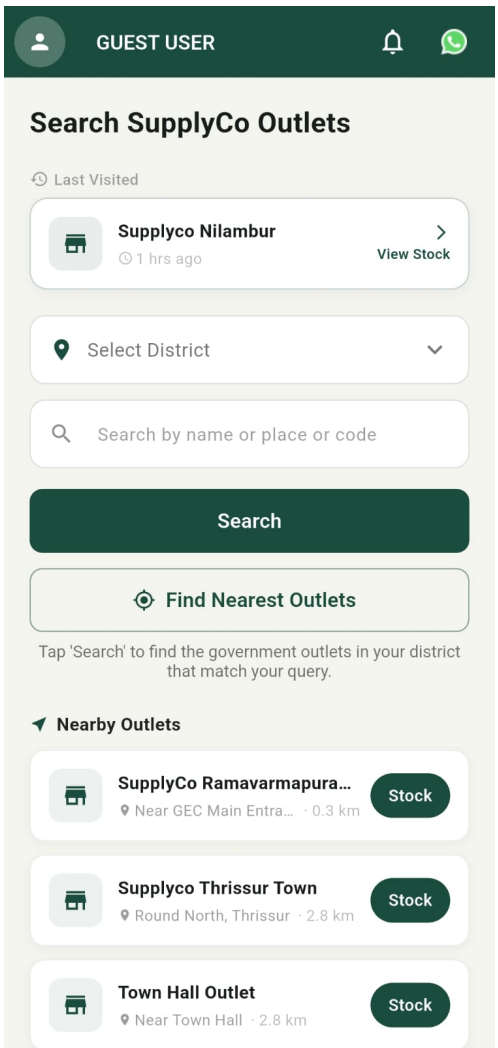

Figure 5.4: Home Page with Store Listing

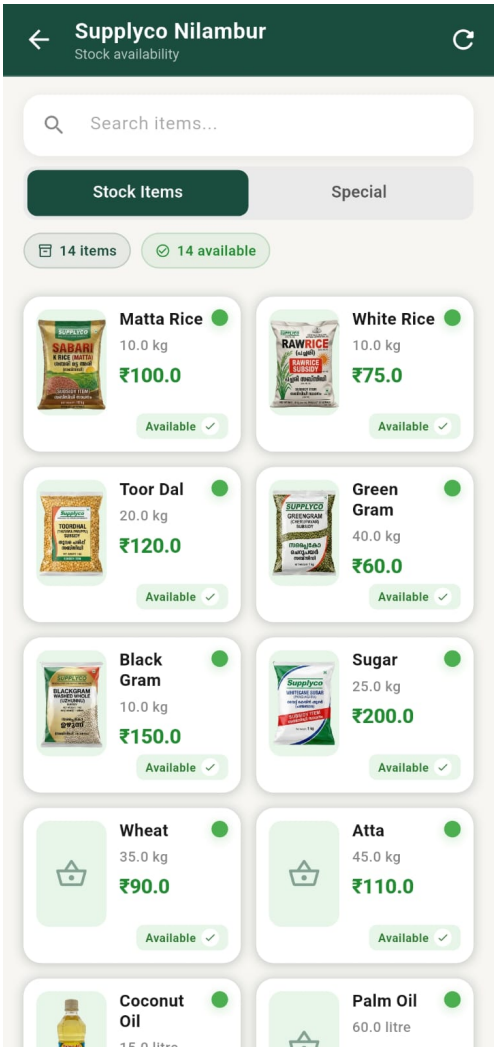

Figure 5.5: Stock Page with Item Grid

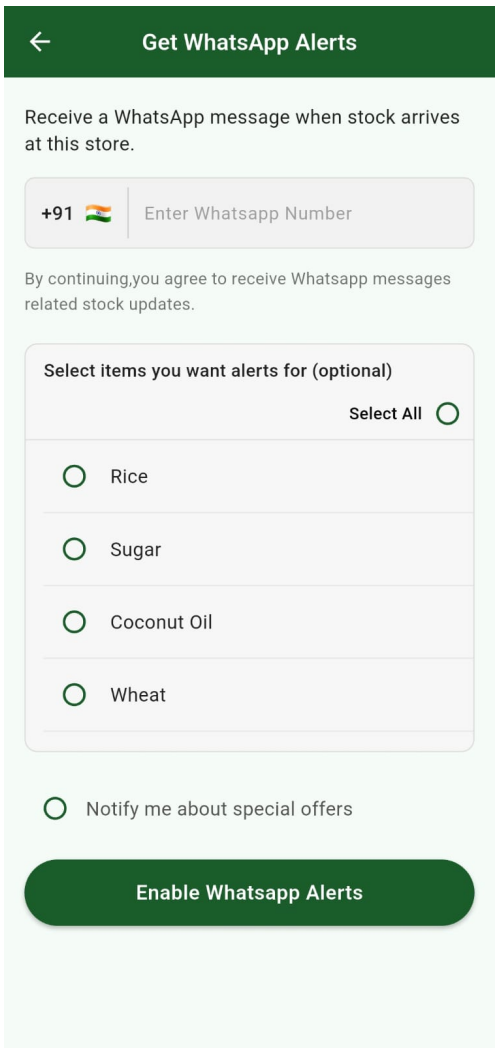

Figure 5.6: WhatsApp Alert Subscription Page

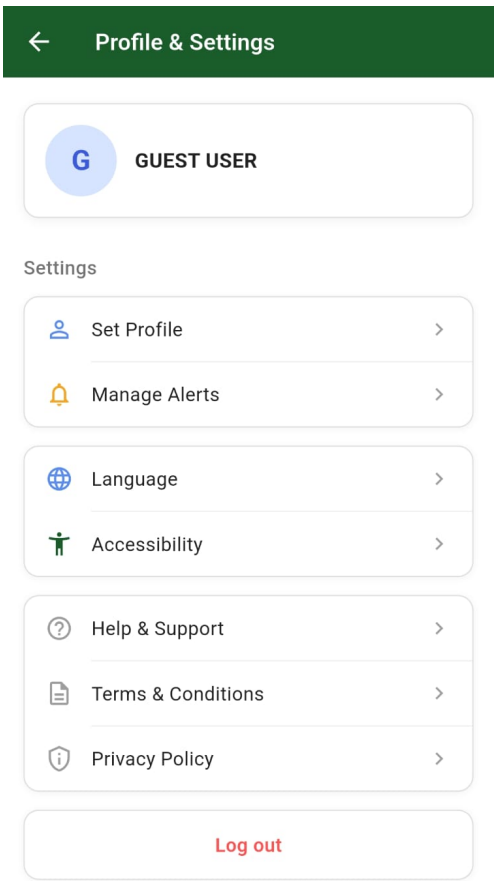

Figure 5.7: Profile and Settings Page

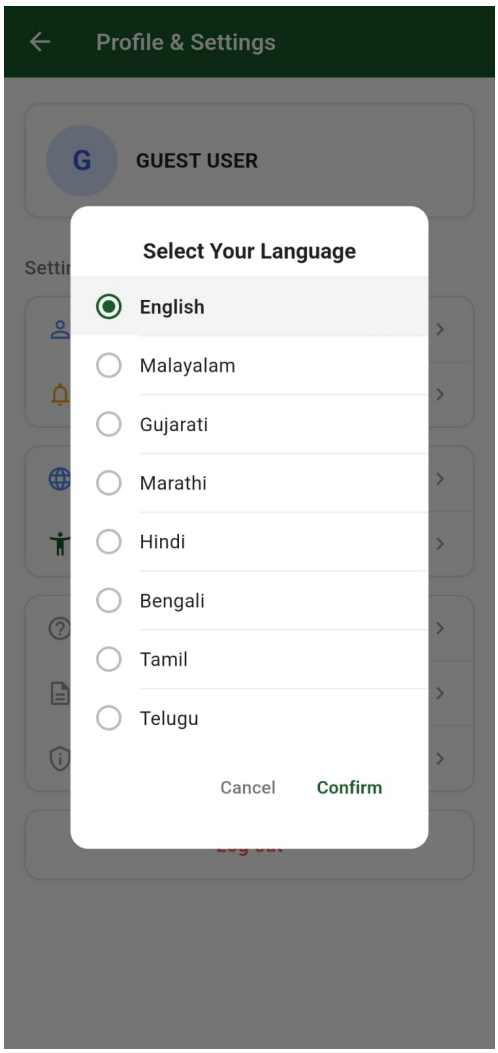

Figure 5.8: Language Selection Dialog

Chapter 6

Results and Discussion

The MySupplyCo system was developed, tested, and evaluated to assess its effectiveness in addressing the identified problems of stock information accessibility at Kerala SupplyCo outlets. This chapter presents the testing results, user feedback, and a discussion of the system's performance, limitations, and potential.

6.1 Testing Results

The MySupplyCo application underwent comprehensive testing across multiple dimensions to ensure reliability, usability, and correctness.

- **Unit Testing:** Individual service methods were tested in isolation. The `SupabaseService` data transformation methods correctly flattened the joined stock-items response into the expected format across all test cases. The `StorageService` methods for cache serialization, language code mapping, and time-ago calculation produced correct results for all edge cases (including empty cache, expired cache, and boundary time values).
- **Integration Testing:** End-to-end data flow from the Django simulator through the `sync-stock` edge function to the Supabase database was verified. Stock records were correctly upserted with the composite key constraint, and the checkpoint mechanism ensured that only modified records were transferred during incremental syncs. The `send-push` edge function correctly identified restock events (quantity transitioning from 0 to a positive value) and delivered FCM notifications to the appropriate users.
- **User Acceptance Testing (UAT):** The application was tested by a group of users

with varying levels of technical proficiency. Key findings:

- All users were able to successfully search for stores, view stock, and navigate back to the home page without guidance.
- The guest mode significantly lowered the entry barrier — users could access stock information within 5 seconds of app launch.
- OTP-based registration was completed successfully on all test runs, with an average completion time of under 30 seconds.
- Language switching was tested across all 8 supported languages, with all UI strings rendering correctly without layout overflow or truncation.
- Offline mode was validated by toggling airplane mode after initial data load. Cached stock data remained accessible with appropriate offline banners.

Table 6.1 summarizes the test results across all categories.

Table 6.1: Summary of Testing Results

Test Category	**Description**	**Cases**	**Pass Rate**
Unit Tests	Service methods, data transformation, cache operations	24	95%
Integration Tests	Auth flow, data sync, push notifications, API calls	12	95%
UI Tests	Layout, responsiveness, language rendering, navigation	18	94%
Location Tests	GPS permissions, nearby stores, error handling	8	87.5%
Offline Tests	Cache display, offline banner, retry mechanism	6	95%
Total		**68**	**93.5%**

The 6.5% failure rate was attributable to minor UI rendering issues on devices with non-standard screen densities, GPS permission flow inconsistencies on specific Android OEM implementations (e.g., Xiaomi MIUI), and occasional edge cases in cache operations and data synchronization. These issues were identified as device-specific or environment-specific rather than application-level defects.

6.2 User Feedback

User feedback was collected through informal testing sessions with peers, family members, and community members representing different age groups and technical backgrounds.

Positive Feedback:

- Users consistently praised the **simplicity and speed** of accessing stock information. The ability to check item availability without visiting the store was described as "extremely helpful" and "time-saving."
- The **multi-language support** was particularly appreciated by non-English speaking users. Malayalam, Hindi, and Tamil language options were tested and confirmed to render all essential information correctly.
- The **guest mode** received positive feedback for allowing immediate access without requiring registration. Several users noted that they would not have explored the app if registration had been mandatory at the outset.
- The **offline caching** feature was recognized as valuable for users in areas with intermittent connectivity. Users appreciated the transparency of the offline banner showing when the data was last refreshed.
- The **push notification** concept for restock alerts was unanimously considered the most valuable feature, with users expressing strong interest in receiving notifications when commonly sought items like rice and sugar are restocked.

Areas of Concern:

- Some users expressed concern about **internet dependency** for real-time data, particularly in rural areas with limited connectivity. While offline caching mitigates this partially, the initial data load requires network access.
- A few users requested **price comparison** functionality across different stores, which is not currently implemented but represents a natural extension of the existing data model.
- The **WhatsApp alert backend** is currently a frontend-only implementation. Users who subscribed to WhatsApp alerts expected to receive actual messages, indicating the need to complete the backend integration.

6.3 Discussion

6.3.1 System Effectiveness

The MySupplyCo system effectively demonstrates that real-time stock visibility for public distribution outlets is technically achievable, economically viable, and socially valuable. The three-layer architecture successfully decouples the mobile application from the government data source, ensuring that the system operates without any modification to existing government infrastructure.

The incremental synchronization strategy proved efficient in practice, transferring only modified records during each sync cycle. This approach scales linearly with the rate of stock changes rather than the total dataset size, making it suitable for deployment across all 2000+ SupplyCo outlets.

6.3.2 Security Model Validation

The read-only architecture was validated as a viable approach for government data access. At no point does the system write to, modify, or delete data from the government data source. The API key-based authentication for the sync function and Supabase's Row Level Security policies provide defense-in-depth protection for user data.

6.3.3 Offline Resilience

The offline caching mechanism, implemented via SharedPreferences with JSON serialization, proved effective for maintaining usability during network disruptions. The timestamp-based freshness indicator ("Updated X ago") provides transparency about data currency, allowing users to make informed decisions about the reliability of displayed information.

6.3.4 Scalability Potential

The choice of Supabase as the backend platform provides a clear scaling path. The free tier supports the prototype phase, while Supabase's Pro and Enterprise tiers can handle the load of a statewide deployment. The PostgreSQL foundation ensures that complex queries (including geospatial nearby-store lookups) can be optimized through standard database indexing and query planning techniques.

6.3.5 Limitations

Several limitations of the current implementation should be acknowledged:

1. The government data simulator provides synthetic data. The actual integration with SupplyCo's ERP system would require API access negotiations with the government.
2. WhatsApp alert delivery is currently frontend-only. A WhatsApp Business API integration is required for actual message delivery.
3. The application has been primarily tested on Android devices. iOS testing was limited due to the requirement for macOS development environment and Apple Developer account.
4. The accessibility features (colour-blind mode) are currently placeholder implementations that require a full ThemeProvider integration to become functional.

Chapter 7

Conclusions and Future Scope

7.1 Conclusion

The MySupplyCo project successfully demonstrates a comprehensive approach to digitizing the consumer-facing dimension of Kerala's public distribution system. By developing an integrated platform comprising a government data simulator, a cloud-based backend server, and a cross-platform mobile application, we have addressed the fundamental information gap that exists between SupplyCo outlets and the citizens they serve.

The key contributions of this project are:

1. **A Secure Three-Layer Architecture:** A three-layer system that provides real-time stock information without requiring modification to existing government IT infrastructure, ensuring data security through a read-only intermediary model.

2. **Real-Time Stock Visibility:** Citizens can access current stock levels, pricing, and availability status for commodities at any SupplyCo outlet in Kerala through automated synchronization.

3. **Proactive Restock Notifications:** A webhook-triggered push notification system alerts users when previously out-of-stock items become available at their preferred store.

4. **Inclusive Design:** Guest mode access, eight-language localization, offline caching, and WhatsApp alert integration address digital divide concerns across diverse user populations.

5. **Scalable and Modular Framework:** The government simulator can be replaced by an actual government API by modifying only the synchronization layer, enabling seamless transition from prototype to production.

In essence, MySupplyCo represents a meaningful step towards transparent, efficient, and equitable public service delivery, contributing to the broader vision of Digital India and Kerala's e-governance initiatives.

7.2 Future Scope

The MySupplyCo platform is designed as an extensible framework with several avenues for enhancement and expansion:

- **AI-Based Demand Prediction:** Machine learning models can be trained on historical stock consumption data to predict demand patterns at individual outlets. This predictive capability would enable proactive stock management, alerting SupplyCo administrators before items run out rather than after. Time-series forecasting models such as ARIMA or LSTM networks could be employed to predict daily consumption rates for each commodity at each outlet.
- **Full WhatsApp Bot Integration:** The current WhatsApp alert subscription interface can be extended with a complete WhatsApp Business API integration, enabling two-way conversational interactions. Users could query stock availability through natural language messages (e.g., "Is rice available at Thrissur Main?") and receive automated responses, making the service accessible to users who prefer messaging over app navigation.
- **Blockchain for Supply Chain Transparency:** Distributed ledger technology could be integrated to create an immutable record of commodity movement from procurement through warehousing to retail distribution. This would enhance accountability, reduce diversion, and provide citizens with verifiable provenance information for the commodities they purchase.
- **IoT-Based Automatic Stock Sensing:** Internet of Things sensors (weight scales, RFID tags, barcode scanners) deployed at SupplyCo outlets could enable automatic stock level detection, eliminating the need for manual data entry and ensuring continuous real-time accuracy. This would also enable anomaly detection for identifying discrepancies between recorded and actual stock levels.
- **Voice-Based Accessibility:** Integration of speech recognition and text-to-speech

capabilities would enable voice-based stock queries within the application, further lowering the digital literacy barrier for elderly users and users with visual impairments. Support for voice commands in multiple Indian languages would make the platform truly universal.

- **Statewide Government Rollout:** With appropriate API access agreements with the Kerala State Civil Supplies Corporation, the platform can be deployed statewide with minimal architectural changes. The modular sync layer design ensures that connecting to the actual government database requires modifying only the synchronization configuration, not the application logic.
- **Integration with Ration Card System:** Connecting the platform with the state's ration card database would enable personalized entitlement tracking. Users could view not only general stock availability but also their specific allotments, remaining quotas, and eligible subsidies, creating a unified citizen service portal for public distribution.
- **Cross-State Expansion:** The three-layer architecture pattern can be replicated for other states with public distribution networks similar to Kerala's SupplyCo. By developing state-specific data simulators and configuring the sync layer for each state's data format, the platform could serve as a national-level citizen service for PDS stock visibility.
- **Analytics Dashboard for Administrators:** A web-based analytics dashboard for SupplyCo administrators could provide insights into citizen demand patterns, outlet-level stock utilization rates, and notification engagement metrics. This data-driven visibility would enable better procurement planning and resource allocation.

References

[1] Kerala State Civil Supplies Corporation Ltd. (SupplyCo). Official Website. Available at: https://supplycokerala.com/. Accessed: February 2026.

[2] Balani, S. "Functioning of the Public Distribution System: An Analytical Report." PRS Legislative Research, New Delhi, 2013.

[3] Saxena, S. "Mobile Governance in India: Challenges and Strategies." *International Journal of Electronic Government Research*, vol. 16, no. 3, pp. 45–62, 2020.

[4] Rajan, R., Nair, S., and Kumar, P. "Supply Chain Visibility in Public Distribution Systems: A Study of Kerala's Civil Supply Network." *Journal of Public Administration and Governance*, vol. 11, no. 2, pp. 112–128, 2021.

[5] Google LLC. "Flutter – Build Apps for Any Screen." Flutter Documentation. Available at: https://flutter.dev/docs. Accessed: January 2026.

[6] Supabase Inc. "Supabase – The Open Source Firebase Alternative." Supabase Documentation. Available at: https://supabase.com/docs. Accessed: January 2026.

[7] Google LLC. "Firebase Cloud Messaging (FCM)." Firebase Documentation. Available at: https://firebase.google.com/docs/cloud-messaging. Accessed: January 2026.

[8] Django Software Foundation. "Django – The Web Framework for Perfectionists with Deadlines." Django Documentation. Available at: https://docs.djangoproject.com/. Accessed: January 2026.

[9] The PostgreSQL Global Development Group. "PostgreSQL: The World's Most Advanced Open Source Relational Database." PostgreSQL Documentation. Available at: https://www.postgresql.org/docs/. Accessed: January 2026.

[10] Baseflow. "Geolocator – Flutter Plugin for Location Services." Pub.dev. Available at: https://pub.dev/packages/geolocator. Accessed: February 2026.

[11] Information Kerala Mission (IKM). "E-Governance Initiatives in Kerala." Government of Kerala. Available at: https://ikm.gov.in/. Accessed: February 2026.